

CCPL SAST/DAST Tool

Shared Project Onboarding and Current Implementation Brief

Current focus: Web SAST on DVWA source code

Environment: Windows Server 2025 VM, 16 GB RAM, 8 vCPUs

Local AI stack: Ollama + Qwen3 8B (Q4_K_M)

Read this document first. It explains what we are building, why the design was chosen, how the pipeline works, and what the team should implement first.

Prepared for shared team development and supervisor review

1. What We Are Building

We are **building** a locally hosted, AI-assisted security testing proof-of-concept. The complete design supports four fixed pipelines:

Target	Test Type	Planned Tooling
DVWA	Web SAST	Semgrep + local LLM review
DVWA	Web DAST	OWASP ZAP + local LLM review
DIVA	Mobile SAST	JADX/decompiled output + local LLM review
DIVA	Mobile DAST	MobSF/runtime data + local LLM review

Current implementation boundary: we are only implementing Web SAST first. The other three pipelines remain part of the final architecture, but they must not delay the first working demo.

2. Why We Are Building It

- **Demonstrate an end-to-end local security pipeline** from target selection to a reviewed report.
- **Reduce false positives** by reviewing scanner findings in context instead of copying raw tool output directly into the final report.
- **Protect source code and scan data** by keeping scanners, model weights, inference, and reports on local infrastructure.
- **Build one reusable workflow** so the same normalization, AI review, and report stages can later support web and mobile testing.

3. How the System Works

The scanner and the LLM have different responsibilities. Semgrep identifies potential code patterns and records evidence. The LLM reads the finding together with relevant source-code context, explains the risk, and decides whether the finding is plausible.

A second LLM pass is required. It independently reviews the first assessment and classifies the finding as confirmed, rejected, or requiring manual review.

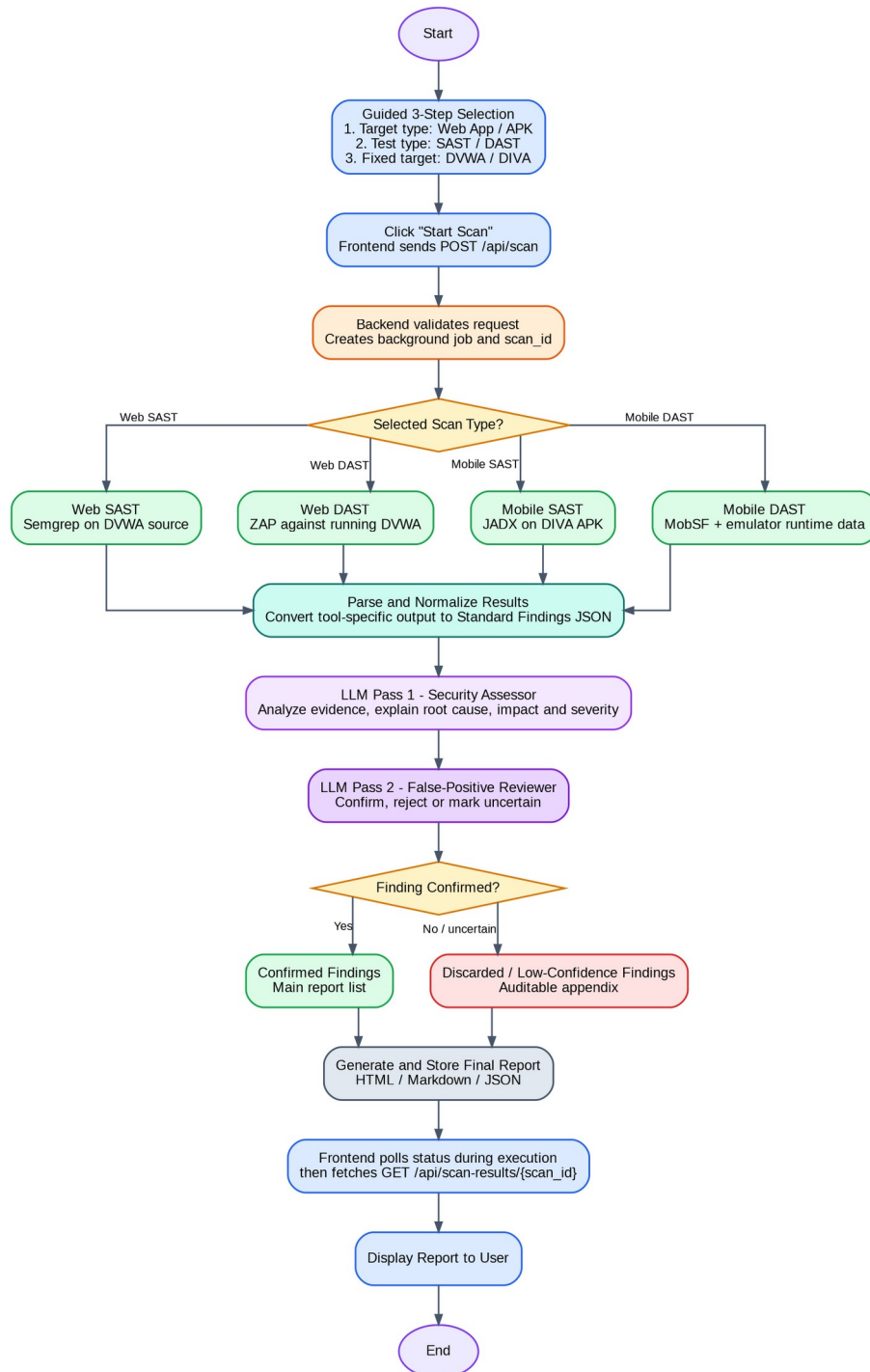
DVWA source -> Semgrep -> Raw JSON -> Normalizer + source context -> Qwen3 assessor
-> Qwen3 reviewer -> Confirmed / discarded -> Report

4. Current Technical Decisions

Area	Decision	Reason
Environment	Windows Server 2025 VM - 16 GB RAM, 8 vCPUs	One controlled local environment for development and demonstration.
Frontend	HTML, CSS, Vanilla JavaScript	The interface is small; a complex framework is not required.
Backend	Python + FastAPI	Fits scanner orchestration, JSON processing, Ollama integration, and later status APIs.
SAST scanner	Semgrep	Produces structured source-code findings and supports local rules.
LLM runtime	Ollama	Simple local model management and a backend-friendly REST API.
Model	Qwen3 8B	Balanced code understanding, reasoning quality, and local resource use.
Quantization	Q4_K_M	Reduces RAM and disk use while retaining practical output quality.
Storage	Local JSON, Markdown, and HTML files	A database is unnecessary for the first proof-of-concept.
Execution	One heavy scan at a time	Avoids CPU and memory contention on the local VM.

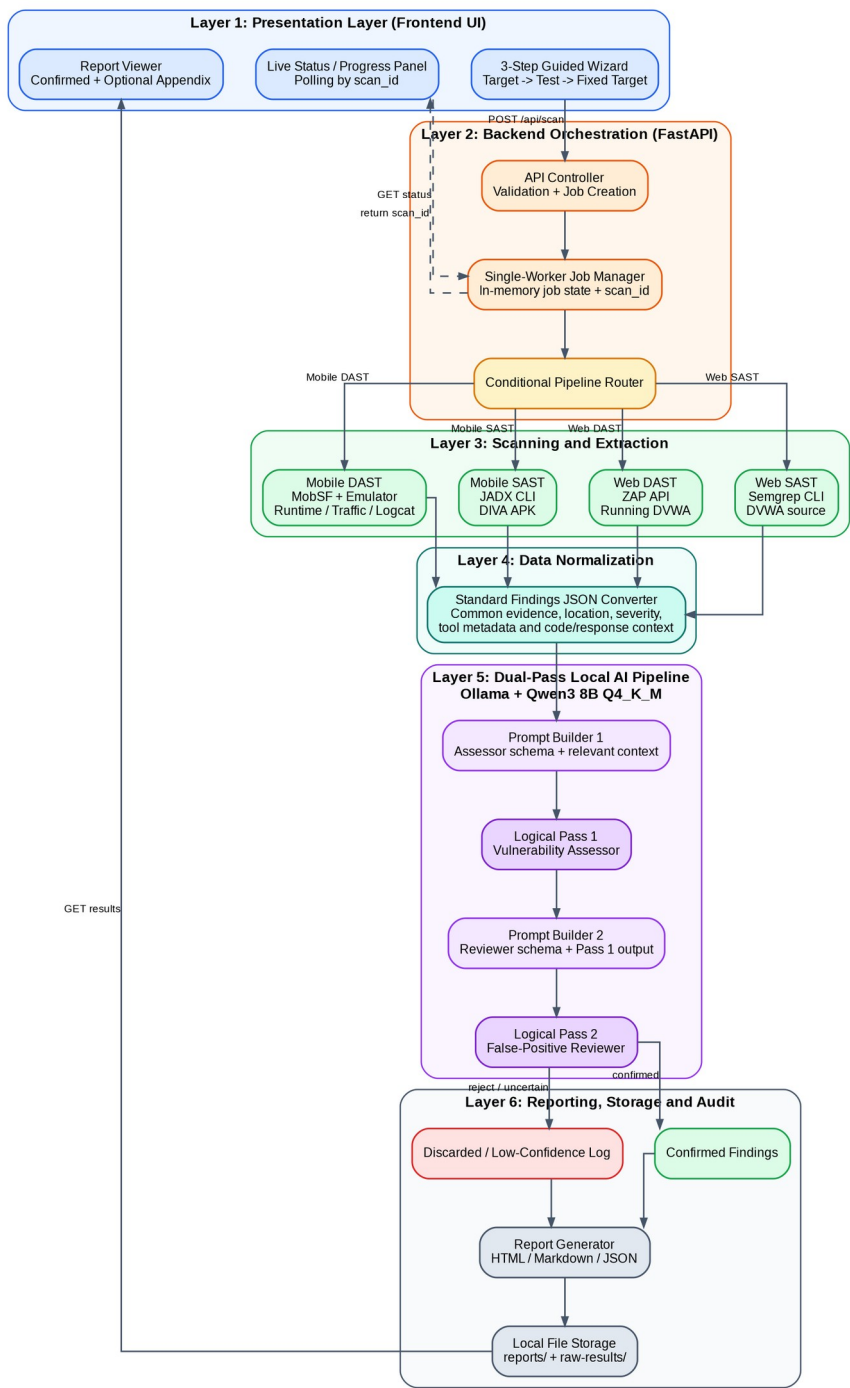
5. Final Workflow

This diagram shows the full business flow. For the current sprint, only the Web SAST branch is being implemented.



6. System Architecture

The **backend is the orchestrator**. It receives the selected scan type, invokes the correct tool, normalizes the result, calls the two local LLM stages, stores the output, and returns status/report data to the frontend.



7. First Implementation Milestone

The first demo must prove one complete pipeline:

1. DVWA source code is available inside the VM.
2. Semgrep runs locally and generates a JSON result file.
3. Findings are converted into a common internal format.
4. Relevant source-code lines are attached to each finding.
5. Qwen3 8B performs the first security assessment through Ollama.
6. The second review pass confirms or rejects each finding.
7. A local Markdown or HTML report is generated.
8. No cloud LLM or cloud scanner is used.

8. What Is Not Part of the First Sprint

- **Web DAST, OWASP ZAP automation, or live DVWA exploitation**
- **DIVA, JADX, MobSF, Frida, or Android emulator integration**
- **User accounts, login, database, or scan-history dashboard**
- **Multiple heavy scans running at the same time**
- **A complex frontend framework**
- **Automatic PDF generation before the core pipeline works**

9. Team Working Agreement

Both members are expected to understand the full pipeline. Primary ownership divides the work, but it does not remove shared responsibility for testing, explanation, and the final demonstration.

- Use one shared repository and small, clear commits.
- Agree on API and JSON contracts before building dependent modules.
- Do not edit the same large file at the same time without coordination.
- Record finalized decisions in the shared documentation.
- Do not commit credentials, tokens, passwords, or private company information.
- Clearly mark work as planned, implemented, tested, or verified.

10. Immediate Next Step

Set up the fresh VM and validate the tools independently before writing the full application.

The first checkpoint is complete when Git, Python, VS Code, Semgrep, Ollama, Qwen3 8B, DVWA source, and a Semgrep JSON scan all work.